Agentic AI Learning Notes

ReAct Pattern · Memory in AI · Mem0 · Mallow Architecture · MCP & JSON-RPC · Feature Engineering · LLM Fine-Tuning · Embedding Model Selection

Generated: May 28, 2026

Grand Azure Bay Hotel Project — Mallow

Table of Contents

Chapter 1 — ReAct Pattern in Agentic AI

ReAct (Reasoning + Acting) is a pattern from the 2022 paper *'ReAct: Synergizing Reasoning and Acting in Language Models'* (Yao et al., Princeton/Google). Instead of just thinking OR calling tools, the LLM interleaves both in a loop.

1.1 The Core Loop

```
Thought → Action → Observation → Thought → Action → ... → Answer

Thought:      LLM reasons about what to do next (chain-of-thought scratchpad)
Action:       LLM calls a tool (search, calculator, database, API)
Observation:  Tool output fed back into context
Repeat until the model emits a final Answer
```

1.2 Why It Works

Pure reasoning (CoT) hallucinates — the model cannot ground itself in real data. Pure acting (tool-calling) lacks planning. ReAct combines both: reasoning guides which tool to call, and tool results correct the reasoning.

Approach	Problem	ReAct Fix
Pure CoT (thinking only)	Hallucinates facts	Grounded by tool observations
Pure tool-calling	No planning or strategy	Thoughts plan each action
ReAct	—	Best of both worlds

1.3 Concrete Example Trace

```
Question: What is the population of the capital of France?

Thought:     I need the capital of France first, then its population.
Action:      search("capital of France")
Observation: Paris

Thought:     Now I need Paris's population.
Action:      search("Paris population")
Observation: ~2.1 million

Thought:     I have all the information needed.
Answer:      The population of Paris (capital of France) is approximately 2.1 million.
```

1.4 In Practice Today

Most modern agent frameworks — LangChain agents, LlamaIndex, the tool_use loop in Anthropic/OpenAI APIs — are ReAct under the hood. The Mallow hotel assistant uses a ReAct loop in **app/agent.py**: the LLM decides to call retrieve_hotel_info or a reservation tool, gets back a result, and continues until it can give a final answer.

■ *Memory + ReAct = powerful. Memory lets the agent look up past experiences as a tool action, turning a stateless loop into one that accumulates expertise.*

Chapter 2 — Memory in Agentic AI

LLMs are stateless — each API call starts fresh. Memory systems give agents continuity, personalization, and the ability to operate over long horizons beyond a single context window.

2.1 Two Axes of Memory

By Duration

Type	Lifespan	Example
Short-term	One session/conversation	Current chat history
Long-term	Across sessions	'User is a data scientist'

By Content (Cognitive Science Classification)

Type	What It Stores	Example
Semantic	Facts & knowledge	'Paris is the capital of France'
Episodic	Past experiences/events	'On 2026-03-01, helped user fix bug X'
Procedural	How to do things / skills	'When asked for tests, use pytest'
Working	Active scratchpad	Current reasoning steps in a ReAct loop

2.2 Implementation Patterns

- **Conversation Buffer**: Append every message. Simple but hits context limit.
- **Sliding Window**: Keep last N messages. Cheap but loses old context.
- **Summarization Memory**: Periodically compress old messages into a summary.
- **Vector Store / RAG**: Store as embeddings; retrieve top-K relevant per turn. Scales to millions.
- **Structured / File-based**: Discrete files with metadata (type, description, content). Inspectable.
- **Knowledge Graph**: Entities + relationships (Zep, Mem0 graph mode). Great for social context.

2.3 The Memory Lifecycle

```
Encode → Store → Retrieve → Update → Forget

Encode:   What's worth saving? Raw text, summary, embedding, or structured fields?
Store:    SQL, vector DB, files, or in-memory dict?
Retrieve: Semantic search, keyword, recency, or relevance scoring?
Update:   Overwrite, append, or version?
Forget:   TTL expiry, manual cleanup, or LRU eviction?
```

2.4 Real Frameworks

Framework	Memory Approach
LangChain	ConversationBufferMemory, VectorStoreRetrieverMemory, summarization
LlamaIndex	Document-store + vector retrieval
Mem0	Hybrid: vector + graph + key-value with auto-extraction
Letta (MemGPT)	OS-inspired: main context + external storage with paging
Zep	Temporal knowledge graph for facts that change over time
Claude Code	File-based structured memory (this session uses this!)

2.5 Hard Problems

- What to remember? Not every utterance is worth saving — need a salience filter.
- When to retrieve? Retrieval on every turn is expensive; missing context is costly.
- Conflict resolution: 'User said X yesterday, Y today' — which wins?
- Staleness: Facts decay. ('User works at Acme Corp' — still true 6 months later?)
- Privacy: Memories can leak sensitive info across sessions or users.
- Hallucination from memory: If a memory is wrong, the agent confidently doubles down.

Chapter 3 — How Mem0 Works

Mem0 is an open-source memory layer for AI agents. It automatically extracts, stores, and retrieves distilled facts — storing what was said, not just what was said.

3.1 The Core Pipeline

```
Conversation messages
        ↓
1. EXTRACT  — LLM extracts atomic facts
              e.g., "User is vegetarian", "User lives in Berlin"
        ↓
2. RESOLVE  — Compare new facts vs existing memories
              Decide: ADD / UPDATE / DELETE / NOOP
        ↓
3. STORE    — Write to vector DB + optional graph DB
              with embeddings + metadata (user_id, timestamps)
        ↓
4. RETRIEVE — On next query: semantic search top-K
              Inject relevant memories into system prompt
```

3.2 Resolution — The Smart Part

Before storing, Mem0 retrieves similar existing memories and uses an LLM to decide:

Operation	When Applied
ADD	Fact is genuinely new information
UPDATE	Fact refines/contradicts existing one ('lived in Paris' → 'lives in Berlin now')
DELETE	Fact invalidates an old one ('no longer works at Acme')
NOOP	Fact already stored or not worth keeping

3.3 Minimal Code Example

```
from mem0 import Memory

m = Memory()

# Session 1 — add a conversation
m.add(
    messages=[
        {"role": "user",      "content": "I'm allergic to peanuts."},
        {"role": "assistant", "content": "Got it, I'll keep that in mind."}
    ],
    user_id="alice"
)

# Session 2 — totally different conversation
results = m.search(query="What can Alice eat?", user_id="alice")
# Returns: [{"memory": "User is allergic to peanuts", "score": 0.89}]
```

3.4 Memory Scoping

Mem0 isolates memories by ID to prevent leakage between users:

- **user_id** — facts about a specific end-user
- **agent_id** — facts an agent has learned
- **run_id** — facts scoped to one session/run

3.5 Comparison with Alternatives

System	Approach	Strength	Weakness
Mem0	LLM-extracted facts + vector + graph	Auto fact distillation, conflict resolution	LLM cost per .add()
Letta (MemGPT)	OS-style paging between tiers	Huge memory, self-managed	Complex, slower
Zep	Temporal knowledge graph	Tracks how facts change over time	Heavier infra
Raw RAG	Embed & retrieve messages	Simple	Noisy, no fact distillation

■ *Mem0 claims ~26% better accuracy and ~91% lower latency vs full-context approaches on the LOCOMO benchmark. Always verify vendor benchmarks independently.*

Chapter 4 — Mallow Project Architecture

Mallow is the **Grand Azure Bay Hotel Assistant** — an AI agent built with Streamlit, Groq LLM (Mixtral-8x7b), ChromaDB, and SQLite. It demonstrates a complete agentic application with RAG + tool-calling + PII guardrails.

4.1 System Architecture

4.2 The Five Tools

Tool	What It Does	Backend
retrieve_hotel_info(query)	Vector search over hotel PDF	ChromaDB
create_reservation(...)	Insert new booking	SQLite
view_my_reservation(id)	Get one booking (owner-scoped)	SQLite
cancel_reservation(id)	Mark booking as cancelled	SQLite
list_my_reservations()	List only the caller's bookings	SQLite

4.3 RAG Pipeline (app/rag.py)

4.4 PII Guardrails (app/db.py)

Every reservation DB query is scoped by a hashed email:

```
# SHA256 hash — guest cannot retrieve someone else's booking
email_hash = hashlib.sha256(email.lower().encode()).hexdigest()[:16]

# Every query includes the hash — no global list tool exists
SELECT * FROM reservations WHERE id = ? AND guest_email_hash = ?
```

4.5 Server-Side Email Injection (app/agent.py:150)

The LLM cannot lie about which guest it is. The Streamlit session email overrides whatever the model puts in tool args:

```
# agent.py line 150-152 — trust the session, not the model
if tool_name in ["create_reservation", "view_my_reservation", ...]:
    tool_input_with_email["guest_email"] = guest_email  # SESSION OVERRIDE
```

4.6 Missing Piece — Long-Term Memory

Mallow has zero long-term memory. A returning guest gets the same blank-slate treatment as a new one. A Mem0 integration would add per-user preferences:

```
# After each conversation turn
mem0.add(messages, user_id=email_hash)

# Before generating a response
prefs = mem0.search(user_message, user_id=email_hash)
# Inject prefs into system prompt → "Book me a room" auto-suggests
# the guest's preferred room type without re-asking
```

4.7 Agentic Concepts in Mallow

Concept	Where in Mallow
ReAct loop	agent.py: while True — Thought (LLM) → Action (tool) → Observation → repeat
Tool routing	tool_choice='auto' — LLM picks RAG vs DB tools
Working memory	self.conversation_history — current session only
Long-term memory	NOT implemented — future enhancement with Mem0
Guardrails	System prompt + missing 'list-all' tool + email session override

Chapter 5 — MCP Servers vs Normal Tool Calls

5.1 Key Differences

Dimension	Normal Tool Calls	MCP Servers
What	Functions defined inside your app	External process exposing tools via protocol
Where	Same codebase as your agent	Separate process / repo / vendor
Coupling	Tightly coupled to one app	Reusable across many agents/apps
Discovery	Hardcoded in agent code	Discovered dynamically at runtime
Analogy	A function in your code	A microservice / plugin

5.2 Normal Tool Call Flow (Mallow today)

```
LLM decides: call retrieve_hotel_info(query="check-in time")
    ↓
Agent code: parses tool_call, calls tool_executor.execute_tool(...)
    ↓
ToolExecutor.retrieve_hotel_info() runs IN-PROCESS (same Python app)
    ↓
Returns result → LLM continues
```

5.3 MCP Tool Call Flow

```
1. Agent starts → connects to MCP server (stdio pipe or HTTP)
2. Agent calls server.list_tools() → receives tool schemas dynamically
3. LLM decides: call "github.create_issue"
4. Agent forwards call to MCP server via JSON-RPC message
5. MCP server executes against GitHub API, returns result
6. Agent passes observation to LLM
```

5.4 The Three MCP Primitives

Primitive	Purpose	Example
Tools	Actions the LLM can invoke	create_pull_request, run_sql
Resources	Read-only data the LLM can fetch	A file, a DB row, a Confluence page
Prompts	Reusable prompt templates	'Review this PR for security issues'

5.5 When to Use Which

Use Normal Tools When	Use MCP Servers When
Tools are tightly bound to app logic	Tool surface is reusable across agents

Building a one-off agent	Want users to swap in capabilities
Performance matters (in-process is faster)	Need language-agnostic tools
Tools need direct app state access	Security team owns tools, app team consumes

■ *The LLM sees identical JSON tool schemas in both cases — it has no idea whether the tool runs in-process or via an MCP subprocess. MCP is plumbing between your agent and the tool implementation, not between LLM and tools.*

Chapter 6 — JSON-RPC and MCP Protocols

6.1 JSON-RPC 2.0

JSON-RPC is a tiny, language-agnostic protocol for calling functions on a remote process using JSON messages. The entire spec fits on one page — deliberately minimal.

The Four Message Types

```
// 1. REQUEST (expects a response)
{"jsonrpc": "2.0", "method": "subtract", "params": {"a": 42, "b": 23}, "id": 1}

// 2. SUCCESS RESPONSE
{"jsonrpc": "2.0", "result": 19, "id": 1}

// 3. ERROR RESPONSE
{"jsonrpc": "2.0", "error": {"code": -32601, "message": "Method not found"}, "id": 1}

// 4. NOTIFICATION (fire-and-forget — no "id" field, no response expected)
{"jsonrpc": "2.0", "method": "log_event", "params": {"event": "user_clicked"}}
```

Key Properties

- Transport-agnostic: works over stdio, HTTP, WebSocket, TCP
- Bidirectional: either side can send requests (not just client→server)
- Stateless: each message is self-contained
- The id field matches requests to responses — many calls can be in flight simultaneously
- Batch support: send an array of requests in one shot

6.2 MCP (Model Context Protocol)

MCP is built on top of JSON-RPC 2.0. It defines which methods exist, what their params/results look like, and how the connection lifecycle works. Think of it as 'USB-C for AI tools' — write once, plug into any compliant agent.

Connection Lifecycle

MCP Transports

Transport	Use Case	How It Works
stdio	Local servers (subprocess)	Client spawns server as child process; messages on stdin/stdout
HTTP + SSE	Remote servers	Client POSTs requests; server pushes responses via Server-Sent Events
Streamable HTTP	Remote, simpler	Single HTTP endpoint, request/response or streaming

6.3 The Full Protocol Stack

```
Layer 4 — Application semantics (MCP)
          "tools/list, tools/call, resources/read, prompts/get"
                    ▲
Layer 3 — RPC framing (JSON-RPC 2.0)
          {method, params, id, result, error}
                    ▲
Layer 2 — Transport
          stdio pipes / HTTP / SSE / WebSocket
                    ▲
Layer 1 — Encoding
          UTF-8 JSON
```

■ *MCP's value is not technical novelty — JSON-RPC was already fine. The value is the shared standard so tools become portable across agents. mcp__confluence__getPage and mcp__playwright__* in this session are real MCP servers.*

Chapter 7 — Feature Engineering & Preprocessing

When all features in a dataset are numerical, the preprocessing pipeline follows a structured sequence of steps before training a machine learning model.

7.1 Pipeline Overview

```
Raw Numerical Data
        ↓
1. Understand the data     (EDA — distributions, correlations, missing counts)
        ↓
2. Handle missing values   (imputation strategy depends on missingness pattern)
        ↓
3. Handle outliers         (detection + treatment based on model type)
        ↓
4. Feature scaling         (standardization or normalization)
        ↓
5. Feature transformation  (fix skew, binning)
        ↓
6. Feature creation        (interactions, ratios, aggregations)
        ↓
7. Feature selection       (remove irrelevant/redundant features)
        ↓
   Model-Ready Data
```

7.2 Missing Values

Missingness Patterns

Pattern	Name	Meaning
Missing randomly	MCAR	Safe to impute straightforwardly
Missing depends on other features	MAR	Impute carefully using related columns
Missing depends on the missing value itself	MNAR	May need special handling

Imputation Strategies

Strategy	When to Use
Drop rows	Very few missing (<1%), MCAR
Drop column	>50-60% missing — probably not useful
Mean imputation	Low skew, no outliers
Median imputation (default)	Skewed distribution or outliers present
KNN imputation	Missingness correlates with other features
Add 'was_missing' flag	Missingness itself is a predictive signal

7.3 Outlier Detection and Treatment

Detection Method	How	When
Z-score	|z| > 3 = outlier	Normal-ish distributions
IQR rule	< Q1-1.5×IQR or > Q3+1.5×IQR	Skewed distributions
Isolation Forest	ML-based anomaly detection	High-dimensional data

Treatment	When
Remove rows	Outlier is a data entry error
Winsorize (clip to 1st/99th percentile)	Preserve row, limit extreme values
Log transform	Compress the scale of right-skewed features
Keep it	Tree models (RF, XGBoost) handle outliers naturally

7.4 Feature Scaling

Method	Formula	Use When	Sensitive To Outliers
StandardScaler	(x - mean) / std	Normal dist, linear/SVM/NN/PCA	Yes
MinMaxScaler	(x - min) / (max - min)	Need [0,1] bounds, neural nets	Very yes
RobustScaler	(x - median) / IQR	Significant outliers present	No

■ *Tree-based models (Decision Tree, Random Forest, XGBoost, LightGBM) do NOT need scaling. Linear models, SVM, KNN, and neural networks DO.*

7.5 Feature Transformation (Fix Skew)

Transform	Formula	For What Skew
Log	log(x + 1)	Right-skewed (long right tail)
Square root	sqrt(x)	Moderately right-skewed
Box-Cox	auto power transform	Right-skewed, requires x > 0
Yeo-Johnson	generalized Box-Cox	Works with zeros and negative values
Reciprocal	1/x	Extreme right skew

7.6 Feature Creation

- **Interaction features**: df['area'] = df['length'] * df['width']
- **Ratios**: df['debt_to_income'] = df['debt'] / (df['income'] + 1)
- **Polynomial features**: PolynomialFeatures(degree=2) — x^2, x1*x2, etc.

- **Group aggregations**: mean, std, max, count per group (user, category)
- **Domain features**: BMI = weight/height², velocity = distance/time

7.7 Feature Selection

Method	What It Does	When to Use
Variance threshold	Remove near-zero variance features	Always as first pass
Correlation filter	Drop one of any pair with r > 0.95	Highly correlated datasets
SelectKBest (f_regression)	Statistical score vs target	Linear relationships
Mutual information	Non-linear relevance to target	Any relationship type
RF feature importance	Model-based ranking	After fitting a tree model
RFE	Recursive elimination via model	Need exact feature count
PCA	Compress to orthogonal components	Hundreds of features, ok losing interpretability

7.8 Golden Rules

- Always fit on train, transform on test — prevents data leakage
- Scale AFTER train/test split — scaler fitted on test leaks test distribution
- Treat outliers differently per model type — trees don't care; linear models do
- Log-transform before scaling — fix the shape before normalizing the range
- Domain knowledge > automated methods — a good ratio beats 10 polynomial features
- Less is more — 20 strong features beat 200 weak ones
- Use a sklearn Pipeline — ensures identical transformations on train and test

Chapter 8 — How to Fine-Tune an LLM

8.1 Should You Fine-Tune?

Try these in order before reaching for fine-tuning:

```
1. Zero-shot prompting
2. Few-shot prompting
3. Chain-of-thought prompting
4. RAG (Retrieval-Augmented Generation)
   ↓
Still not good enough? → Fine-tune
```

Fine-tune WHEN	Don't fine-tune WHEN
Model needs new style/tone	You just need to inject facts (use RAG)
Deep domain knowledge needed	Fewer than ~100 labeled examples
Consistent output format required	Base model already works with a good prompt
Latency: smaller fine-tuned model beats prompting larger one	

8.2 Types of Fine-Tuning

Type	What Updates	GPU Needed	Data Needed
Full fine-tuning	All parameters	A100 80GB for 7B model	Thousands of examples
Partial fine-tuning	Last N layers only	Less, but still large	Hundreds+
LoRA	Small low-rank matrices (~0.1% of params)	RTX 3090 for 7B model	Hundreds
QLoRA	LoRA on 4-bit quantized model	16GB VRAM for 7B model	Hundreds
Prompt Tuning	Only soft input tokens	Minimal	Hundreds

8.3 LoRA — The Key Idea

Instead of updating a full weight matrix W (millions of parameters), learn two small matrices A and B:

```
W_updated = W_original + (A × B)

W original: 4096 × 4096 = 16,777,216 parameters
A + B:      (4096×16) + (16×4096) = 131,072 parameters
                        ↓
             Train only 0.8% of the original parameters!

Key hyperparameters:
  r (rank)       = 8, 16, 32, 64   — higher = more capacity
  lora_alpha     = 2×r typically   — scaling factor
  target_modules = ["q_proj", "v_proj"]  — which attention layers
```

8.4 QLoRA — LoRA on Quantized Model

QLoRA combines 4-bit quantization with LoRA, making 7B model fine-tuning possible on consumer GPUs:

```
7B model in FP16  → ~14GB VRAM  (doesn't fit on consumer GPU)
7B model in 4-bit → ~4GB VRAM
4-bit model + LoRA adapters → ~6GB VRAM  (fits on RTX 3090/4090!)
```

8.5 Fine-Tuning by Approach

Approach	Dataset Format	Use Case
SFT (Supervised)	(input, output) pairs	Most common — teaches a new task
Instruction tuning	(instruction, input, output) triples	ChatGPT-style behavior from base model
DPO	(prompt, chosen, rejected) triples	Teach preferences without RL complexity
RLHF	Human rankings + PPO	Full alignment — used by OpenAI/Anthropic

8.6 Data Guidelines

- Quality over quantity: 100 clean examples > 10,000 noisy ones
- Minimum: ~200-500 examples for style/task adaptation
- Sweet spot: 1,000-5,000 examples for domain knowledge
- Format: use the model's chat template (e.g., [INST]...[/INST] for Mistral)
- Always split train/val: 90/10 is common
- Diverse examples > repetitive ones on the same pattern

8.7 Key Training Hyperparameters

Parameter	Typical Range	Effect
learning_rate	1e-4 to 3e-4	Too high = unstable; too low = no learning
num_train_epochs	2–5	More epochs = overfitting risk
r (LoRA rank)	8, 16, 32, 64	Higher = more capacity, more parameters
batch_size	4–32	Larger = more stable gradients
gradient_accumulation	2–16	Simulate larger batch on small GPU
warmup_ratio	0.03–0.1	Ramp up LR to avoid early instability

8.8 Watch for Catastrophic Forgetting

After fine-tuning, test on general benchmarks (HellaSwag, MMLU, TruthfulQA) to ensure the model hasn't lost its general capabilities while learning the new task. LoRA is less prone to this than full fine-tuning because the base weights remain frozen.

8.9 Decision Guide

Dataset size?
< 200 examples? → Use RAG or few-shot prompting instead
200–5,000? → QLoRA on Mistral-7B or Llama-3-8B
5,000–100,000? → LoRA on 13B–70B, or full fine-tune on 7B
100,000+ + budget → Full fine-tuning or RLHF/DPO

Chapter 9 — Selecting the Right Embedding Model

An embedding model maps text to a vector of numbers. A good model places semantically similar text near each other — even when the words are completely different.

9.1 The Four Key Questions

- What is my task? (retrieval, clustering, classification, re-ranking?)
- What is my domain? (general English, medical, legal, code?)
- What are my constraints? (latency, cost, privacy, storage?)
- What language? (English only, multilingual, cross-lingual?)

9.2 Task Types — Bi-encoder vs Cross-encoder

```
Bi-Encoder (fast — for retrieval):
  Query  → [Encoder] → vector Q
  Doc    → [Encoder] → vector D
  Score  = cosine(Q, D)
  ✓ Pre-compute doc embeddings once
  ✓ Millisecond lookup in vector DB at query time

Cross-Encoder (accurate — for re-ranking):
  [Query + Doc concatenated] → [Encoder] → single score
  ✗ Cannot pre-compute — needs both at inference time
  ✓ Much more accurate relevance scoring
  ✓ Use AFTER bi-encoder retrieval to re-rank top results

Best practice pipeline:
  User query
      ↓
  Bi-encoder → retrieve top 20 candidates (fast)
      ↓
  Cross-encoder → re-rank top 20, keep top 3 (accurate)
      ↓
  LLM gets the 3 best chunks
```

9.3 Model Comparison

Model	Dims	Size	Quality	Cost
all-MiniLM-L6-v2	384	80MB	Good	Free (local)
all-mpnet-base-v2	768	420MB	Better	Free (local)
bge-base-en-v1.5	768	430MB	Very good	Free (local)
bge-large-en-v1.5	1024	1.3GB	Excellent	Free (local)
e5-large-v2	1024	1.3GB	Excellent	Free (local)

text-embedding-3-small	1536	API	Very good	$0.02/1M tokens
text-embedding-3-large	3072	API	Best	$0.13/1M tokens
voyage-large-2	1536	API	Best-in-class	$0.12/1M tokens

9.4 MTEB Benchmark

MTEB (Massive Text Embedding Benchmark) by Hugging Face evaluates models across 56 tasks. For RAG, look at the Retrieval column specifically — don't trust the overall average score.

- bge-large-en-v1.5 — best free local model for retrieval (~54 MTEB retrieval)
- voyage-large-2-instruct — best overall (~60+ MTEB retrieval, API)
- all-MiniLM-L6-v2 — good for its tiny size (~41 MTEB retrieval, Mallow's model)

9.5 Instruction-Based Embeddings

Modern models (BGE, E5, voyage) support task instructions prepended to queries:

```
# BGE / E5 pattern — only the QUERY gets the instruction, not documents
query = "Represent this sentence for searching relevant passages: " + user_query
query_embedding = model.encode(query)
doc_embedding   = model.encode(doc_text)  # no prefix for documents
```

9.6 Empirical Evaluation Steps

- Step 1: Build 20-50 (query, relevant_doc) pairs from your actual domain data
- Step 2: Run each candidate model: encode queries + docs, compute cosine similarity
- Step 3: Measure Recall@1 and Recall@3: does the top result match the expected doc?
- Step 4: Measure latency: queries per second at your target batch size
- Step 5: Pick: quality gap < 5% → take the smaller/faster model

9.7 Applied to Mallow

Mallow currently uses all-MiniLM-L6-v2 (384-dim). For hotel text (everyday English, small doc), this is sufficient. A free upgrade with no infra change:

```
# app/rag.py line 13 — drop-in replacement
# From:
self.model = SentenceTransformer('all-MiniLM-L6-v2')

# To (meaningfully better retrieval, still free/local):
self.model = SentenceTransformer('BAAI/bge-base-en-v1.5')

# And add instruction prefix for BGE models:
def retrieve(self, query, top_k=3):
    query = "Represent this sentence for searching relevant passages: " + query
    query_embedding = self.model.encode(query).tolist()
    ...
```

9.8 Quick Decision Reference

Need	Model
Speed critical, local/free	all-MiniLM-L6-v2 (384-dim, 80MB)
Best free local quality	BAAI/bge-large-en-v1.5 (1024-dim, 1.3GB)
Best overall (API ok)	voyage-large-2 or text-embedding-3-large
Multilingual	multilingual-e5-large
Code search	text-embedding-3-large (API)
Re-ranking after retrieval	bge-reranker-large (cross-encoder)
Domain-specific (medical/legal)	Check MTEB domain tab for specialized models

Quick Reference Summary

Key Concepts at a Glance

Topic	Core Takeaway
ReAct	Thought → Action → Observation loop. Reasoning guides tools; tools correct reasoning.
Memory types	Semantic (facts), Episodic (events), Procedural (skills), Working (scratchpad)
Mem0	LLM extracts atomic facts, resolves conflicts (ADD/UPDATE/DELETE), stores in vector+graph DB
Mallow	ReAct agent: RAG for Q&A, SQLite for reservations, email-hash PII scoping, no long-term memory
MCP vs tools	Normal = in-process function. MCP = separate process over JSON-RPC, reusable across agents.
JSON-RPC	Tiny protocol: {method, params, id} → {result} or {error}. Transport-agnostic.
Preprocessing	Missing→median, outliers→winsorize (linear) or ignore (trees), skew→log, scale→StandardScaler
Fine-tuning	Try RAG first. Then QLoRA (LoRA on 4-bit model, fits ~6GB VRAM). SFT→DPO→RLHF in complexity.
Embeddings	Task matters most. Use MTEB retrieval score. bge-large-en best free local. Test on your own data.

Learning Path Recommendation

- 1. Understand ReAct — it's the foundation of all agentic behavior
- 2. Study the Mallow codebase — app/agent.py is a real ReAct implementation
- 3. Experiment with RAG: try upgrading Mallow's embedding model to bge-base-en-v1.5
- 4. Add Mem0 to Mallow: store guest preferences across sessions
- 5. Build a minimal MCP server wrapping Mallow's tools
- 6. Fine-tune a small model (Mistral-7B with QLoRA) on hotel Q&A; data
- 7. Benchmark your fine-tuned model against RAG — understand when each is better